

راهنمای آموزشی KSC.Observability

مانیتورینگ و متریک آماده‌نصب برای اپلیکیشن‌های **NET Framework** (ASP.NET Web Forms / MVC).
پایه **Prometheus** و **Grafana**.

این سند هم توضیح می‌دهد چه چیزی ساخته شد و چرا، و هم گام‌به‌گام یاد می‌دهد چطور استفاده کنید.

فهرست

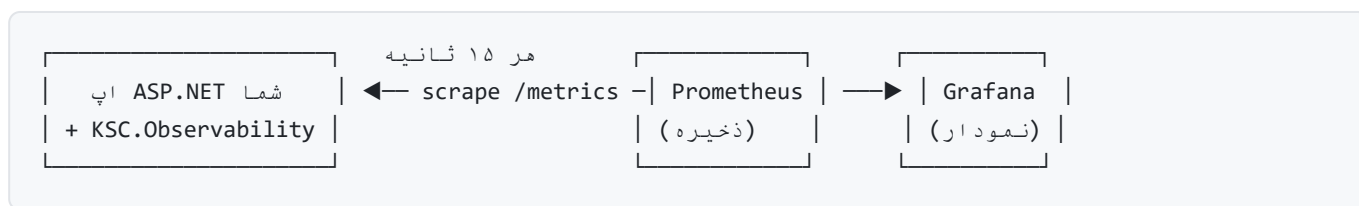
1. مسئله چه بود؟
2. مفاهیم پایه (قبل از شروع بخوانید)
3. معماری پروژه
4. متریک‌هایی که جمع می‌شوند
5. پشت صحنه: چطور کار می‌کند؟
6. نصب در اپلیکیشن واقعی (گام‌به‌گام)
7. تنظیمات
8. راه‌اندازی استک مانیتورینگ
9. اجرای دمو با یک دستور
10. کار با Grafana
11. کوئری‌های کاربردی PromQL
12. امنیت endpoint متریک
13. عیب‌یابی
14. Build و انتشار پکیج
15. سؤالات متداول

۱. مسئله چه بود؟

شما کلی اپلیکیشن **NET Framework** دارید، اما هیچ دیدی نسبت به وضعیت آن‌ها ندارید:

- نمی‌دانید همین الان چند کاربر همزمان از یک سیستم استفاده می‌کنند.
- نمی‌دانید یک سیستم چقدر CPU و RAM مصرف می‌کند.
- نمی‌دانید اصلاً یک اپ بالا هست یا پایین.
- نمی‌دانید کندی از کجاست و نرخ خطا چقدر است.

راه حل استاندارد صنعتی برای این مسئله، **Observability** (رصدپذیری) با Prometheus + Grafana است. ما این را به شکل یک پکیج **NuGet آماده نصب** پیاده کردیم: نصبش می کنید، تمام.



۲. مفاهیم پایه

برای اینکه ادامه برایتان واضح باشد، چند مفهوم را سریع مرور کنیم:

Prometheus چیست؟

یک پایگاه دادهٔ سری زمانی (Time-Series DB) که **خودش به صورت دوره‌ای** (مثلاً هر ۱۵ ثانیه) به اپ شما سر می زند و یک صفحهٔ متنی به نام `/metrics` را می خواند و مقادیر را ذخیره می کند. به این مدل **Pull** می گویند (برخلاف مدل Push که اپ خودش داده می فرستد). مزیت Pull: اپ شما ساده می ماند و Prometheus می داند هر هدف بالا هست یا نه (اگر scrape نشد، یعنی down).

Grafana چیست؟

ابزار نمایش داشبورد. به Prometheus وصل می شود و با زبان **PromQL** کوئری می زند و نمودار/هشدار می سازد.

فرمت متریک Prometheus

یک متن ساده است. هر خط یک نمونه است:

```
ksc_active_users{service="billing",instance="SRV01",env="production"} 24
```

نام متریک ————— لیبلها ————— مقدار

انواع متریک

نوع	یعنی چه	مثال در پروژه
Counter	فقط زیاد می شود (شمارنده)	تعداد کل ریکوئست ها
Gauge	بالا/پایین می رود (عقربه ای)	کاربران فعال، مصرف RAM
Histogram	توزیع مقادیر در بازه ها	زمان پاسخ ریکوئست ها

لیبل و «Cardinality»

لیبل‌ها (مثل `code` , `method`) به متریک بُعد می‌دهند. اما **مراقب باشید**: اگر یک لیبل مقادیر بی‌نهایت بگیرد (مثلاً مسیر کامل URL با id داخلش)، تعداد سری‌ها منفجر می‌شود و Prometheus کند/پر می‌شود. به این مشکل **High Cardinality** می‌گویند. به همین دلیل ما لیبل `path` را به صورت **پیش‌فرض خاموش** گذاشتیم.

۳. معماری پروژه

پروژه با **Clean Architecture** ساخته شده: لایه‌ها از داخل به بیرون، و وابستگی‌ها فقط رو به داخل.

```
KSC.Observability/
├─ src/
│   ├── KSC.Observability.Abstractions ← (بدون هیچ وابستگی) Options قرارداده‌ها و Core: لایه
│   ├── KSC.Observability.Metrics ← prometheus-net پیاده‌سازی با Infrastructure: لایه
│   └── KSC.Observability.AspNet ← همان پکیجی که نصب می‌کنید Integration: لایه
├─ samples/
│   ├── KSC.Sample.SelfHost ← برای تست سریع (IIS بدون) دمو کنسولی
│   └── KSC.Sample.WebApp ← Visual Studio (برای) ASP.NET نمونه واقعی
├─ tests/KSC.Observability.Tests ← (xUnit) تست‌های واحد
├─ deploy/ ← داشبورد + Prometheus + Grafana (docker-compose)
├─ build/pack.ps1 ← NuGet ساخت پکیج‌های
├─ up.cmd / down.cmd ← بالا/پایین آوردن کل محیط با یک دستور
└─ .github/workflows/ci.yml ← بیلد/تست/پکیج خودکار
```

لایه	پروژه	Target	چرا؟
Core	Abstractions	netstandard2.0	فقط قرارداد و interface؛ به هیچ کتابخانه‌ای وابسته نیست تا قابل تعویض بماند
Infrastructure	Metrics	net472	پیاده‌سازی واقعی متریک‌ها با prometheus-net
Integration	AspNet	net472	اتصال به System.Web (HttpModule، endpoint، ثبت خودکار)

جهت وابستگی: Abstractions → Metrics → AspNet. لایه بیرونی (AspNet) همه را می‌شناسد؛ لایه‌های داخلی هیچ‌چیز بیرونی را نمی‌شناسند. نتیجه: اگر فردا خواستید به جای Prometheus از سیستم دیگری استفاده کنید، فقط لایه Metrics را عوض می‌کنید و قراردادهای (Abstractions) و کد اپ شما دست‌نخورده می‌ماند.

۴. متریک‌هایی که جمع می‌شوند

همه متریک‌ها لیبل‌های `instance` ، `service` و `env` را دارند (پیشوند `ksc` قابل تغییر است).

متریک	نوع	لیبل اضافه	یعنی چه
ksc_active_users	Gauge	—	تعداد کاربران فعال همزمان در بازه زمانی تعیین شده 👤
ksc_http_requests_total	Counter	method , code	تعداد کل ریکوئست‌ها
ksc_http_requests_in_flight	Gauge	—	ریکوئست‌هایی که همین الان در حال پردازش‌اند 🔄
ksc_http_request_duration_seconds	Histogram	method	زمان پاسخ ریکوئست‌ها ⌚
ksc_process_cpu_usage_percent	Gauge	—	درصد CPU (نسبت به یک هسته) 🧠
ksc_process_working_set_bytes	Gauge	—	حافظه فیزیکی در حال استفاده
ksc_process_private_memory_bytes	Gauge	—	حافظه خصوصی پراسس
ksc_process_managed_memory_bytes	Gauge	—	حجم heap مدیریت‌شده GC
ksc_process_threads	Gauge	—	تعداد threadها
ksc_process_handles	Gauge	—	تعداد handleهای سیستم‌عامل
ksc_process_uptime_seconds	Gauge	—	چند ثانیه است که پراسس بالااست
ksc_gc_collections_total	Counter	generation	تعداد GCها به تفکیک نسل 🔄
ksc_build_info	Gauge	version	همیشه ۱؛ نسخه کتابخانه را به عنوان لیبل دارد

۵. پشت صحنه: چطور کار می‌کند؟

۵.۱ ثبت خودکار ماژول (بدون دستکاری web.config)

وقتی پکیج را نصب می‌کنید، این خط در اسمبلی وجود دارد:

```
[assembly: PreApplicationStartMethod(typeof(ObservabilityHttpModule), "OnPreApplicationStart")]
```

ASP.NET این متد را قبل از `Application_Start` صدا می‌زند و ماژول با `DynamicModuleUtility.RegisterModule` به صورت پویا ثبت می‌شود. به همین خاطر نیازی به افزودن `<modules>` در **web.config** ندارید — فقط نصب کافی است.

۵.۲ شمارش کاربران همزمان

در هر ریکوئست (مرحله `PostAcquireRequestState`)، یک «کلید کاربر» استخراج می‌شود: ابتدا `SessionID`، اگر نبود نام کاربر احرازهویت‌شده، و در نهایت IP. این کلید با زمان فعلی در یک دیکشنری ذخیره می‌شود. یک تایمر پس‌زمینه هر چند ثانیه کلیدهای قدیمی‌تر از «پنجره زمانی» (پیش‌فرض ۵ دقیقه) را پاک می‌کند و تعداد باقی‌مانده را در گاج `ksc_active_users` می‌نویسد. نتیجه: تعداد کاربرانی که در ۵ دقیقه اخیر فعال بوده‌اند.

۵.۳ متریک‌های HTTP

- در ابتدای ریکوئست: گاج `in_flight` یک واحد زیاد می‌شود و یک `Stopwatch` شروع می‌شود.
- در انتهای ریکوئست: `in_flight` کم می‌شود، شمارنده کل با لیبل‌های `method / code` زیاد می‌شود و مدت‌زمان در هیستوگرام ثبت می‌شود.

۵.۴ محاسبه CPU

به‌جای `PerformanceCounter` (که روی IIS با نام‌گذاری instance شکننده است)، از اختلاف `Process.TotalProcessorTime` در بازه زمانی استفاده می‌کنیم:

$$\text{CPU\%} = \frac{(\text{زمان واقعی} \times \text{تعداد هسته}) - (\text{زمان پردازنده})}{\Delta}$$

این روش پایدار و دقیق است و به دسترسی خاصی نیاز ندارد.

۵.۵ endpoint متریک

خود ماژول، مسیر `/metrics` را تشخیص می‌دهد و خروجی Prometheus را می‌نویسد و درخواست را همان‌جا کامل می‌کند (نیازی به ثبت Handler جداگانه نیست).

۶. نصب در اپلیکیشن واقعی

گام ۱ — انتشار پکیج روی فید داخلی

پکیج‌ها در این مخزن ساخته می‌شوند (`build/pack.ps1`) و در پوشه `artifacts` قرار می‌گیرند. آن‌ها را روی فید داخلی شرکت (Azure Artifacts، BaGet)، یا یک پوشه اشتراکی) بگذارید تا اپ‌ها بتوانند نصب کنند.

گام ۲ — نصب در اپ

در کنسول NuGet اپ ASP.NET:

```
Install-Package KSC.Observability.AspNetCore
```

همین پکیج به صورت زنجیره‌ای `Microsoft.Web.Infrastructure` و `Metrics`، `Abstractions`، `prometheus-net` را هم می‌آورد.

گام ۳ — (اختیاری) تنظیم نام سرویس

در `Global.asax.cs`:

```
protected void Application_Start(object sender, EventArgs e)
{
    KscObservability.Initialize(options =>
    {
        options.ServiceName = "billing-portal";
        options.Environment = "production";
    });
}
```

اگر این کار را هم نکنید، مازول در اولین ریکوئست خودش از روی `web.config` (یا پیش‌فرض‌ها) مقداردی می‌شود.

گام ۴ — تست

اپ را اجرا کنید و آدرس `/metrics` را باز کنید. باید خروجی متنی متریک‌ها را ببینید. تمام — از این لحظه اپ شما قابل‌مانیتور است.

۷. تنظیمات

دو راه دارید: **کد** (در `Initialize`) یا **بدون کد** (در `web.config`). اگر هر دو باشند، کد برنده است.

از طریق `web.config` (بدون کد)

```
<appSettings>
  <add key="KSC.Observability:ServiceName" value="billing-portal" />
  <add key="KSC.Observability:Environment" value="production" />
  <add key="KSC.Observability:MetricsPath" value="/metrics" />
  <add key="KSC.Observability:ActiveUserWindowSeconds" value="300" />
  <!-- <add key="KSC.Observability:MetricsAccessToken" value="یک-توکن-محرمانه" /> -->
</appSettings>
```

جدول کامل تنظیمات

معنی	پیش فرض	کلید (با پیشوند: KSC.Observability)
لیبل service	dotnet-app	ServiceName
لیبل instance	نام ماشین	InstanceId
لیبل env	production	Environment
پیشوند نام متریک ها	ksc	MetricPrefix
مسیر endpoint	/metrics	MetricsPath
متریک های .../CPU/RAM/GC	true	EnableSystemMetrics
متریک های ریکوئست	true	EnableHttpMetrics
شمارش کاربران فعال	true	EnableActiveUserTracking
بازه نمونه برداری سیستم	5	SystemMetricsIntervalSeconds
پنجره «فعال بودن» کاربر	300	ActiveUserWindowSeconds
افزودن لیبل path (مراقب cardinality!)	false	TrackRequestPath
محافظت endpoint با Bearer Token	—	MetricsAccessToken

۸. راه اندازی استک مانیتورینگ

پوشه `deploy/` یک استک آماده Prometheus + Grafana دارد.

```
cd deploy
docker compose up -d
```

ورود	آدرس	سرویس
admin / admin	http://localhost:3000	Grafana
—	http://localhost:9090	Prometheus

وصل کردن Prometheus به اپ های شما

فایل `deploy/prometheus/prometheus.yml` را ویرایش کنید و آدرس `/metrics` هر اپ را زیر `ksc-dotnet-apps` اضافه کنید:

```
static_configs:
  - targets:
    - app-server-01:80
    - app-server-02:80
  labels:
    team: billing
```

سپس Prometheus را بدون قطعی reload کنید:

```
curl -X POST http://localhost:9090/-/reload
```

سلامت هدف‌ها را در `http://localhost:9090/targets` ببینید (باید `up` باشند).

۹. اجرای دمو با یک دستور

برای دیدن کل چرخه به صورت زنده (بدون نیاز به IIS)، از ریشه پروژه و با **Docker روشن**:

```
.\up.cmd
```

این یک دستور: اپ دمو را build می‌کند، استک را بالا می‌آورد، اپ نمونه را اجرا می‌کند، صبر می‌کند تا همه healthy شوند و مرورگر را روی داشبورد باز می‌کند.

دستور	کاربرد
<code>.\up.cmd</code>	کل دمو (استک + اپ نمونه)
<code>.\up.cmd -NoDemo</code>	فقط استک مانیتورینگ (برای محیط واقعی)
<code>.\up.cmd -NoBrowser</code>	بدون باز کردن مرورگر
<code>.\down.cmd</code>	توقف اپ + استک
<code>.\down.cmd -Volumes</code>	توقف + پاک کردن دیتای ذخیره شده

۱۰. کار با Grafana

- به `http://localhost:3000` بروید (admin / admin؛ بار اول رمز را عوض کنید).
- داشبورد **KSC.Observability — Overview** به صورت خودکار provision شده (پوشه KSC.Observability).
- از منوی **Service** بالای داشبورد، اپ موردنظر را انتخاب کنید (یا All).
- بازه زمانی بالا-راست را روی **Last 15 minutes** و `auto-refresh` را روشن کنید.

پنل‌های داشبورد: کاربران فعال، ریکوئست در حال پردازش، نرخ ریکوئست/خطا، CPU، latency (p50/p95/p99)، حافظه، thread/handle و GC.

داشبورد از فایل ساخته می‌شود (deploy/grafana/dashboards/ksc-overview.json). برای تغییر دائمی، همین فایل را ویرایش کنید (تا با restart از بین نرود).

۱۱. کوئری‌های کاربردی PromQL

```
# کاربران همزمان هر اپ
sum by (service) (ksc_active_users)

# نرخ ریکوئست (در ثانیه)
sum by (service) (rate(ksc_http_requests_total[5m]))

# نسبت خطا (5xx)
sum by (service) (rate(ksc_http_requests_total{code=~"5.."}[5m]))
/ sum by (service) (rate(ksc_http_requests_total[5m]))

# صدک ۹۵ زمان پاسخ
histogram_quantile(0.95, sum by (le) (rate(ksc_http_request_duration_seconds_bucket[5m])))

# نشده scrape آیا اپ پایین است؟
up{job="ksc-dotnet-apps"} == 0

# مصرف حافظه (مگابایت)
ksc_process_working_set_bytes / 1024 / 1024
```

۱۲. امنیت endpoint متریک

به صورت پیش فرض `/metrics` باز است. اگر اپ روی شبکه غیرامن قرار دارد، یک توکن تعیین کنید:

```
<add key="KSC.Observability:MetricsAccessToken" value="یک-توکن-محرمانه" />
```

سپس در Prometheus همان توکن را بدهید:

```
authorization:
  type: Bearer
  credentials: "یک-توکن-محرمانه"
```

از این به بعد هر درخواست بدون هدر `Authorization: Bearer ...` با کد ۴۰۱ رد می‌شود.

توصیه دیگر: endpoint را در سطح شبکه/فایروال فقط برای IP سرور Prometheus باز بگذارید.

۱۳. عیب‌یابی

نشانه	علت محتمل	راه‌حل
در <code>/targets</code> هدف <code>down</code> با خطای <code>connection refused</code> است	اپ بالا نیست یا پورت اشتباه است	آدرس و پورت <code>/metrics</code> را بررسی کنید
هدف <code>down</code> با <code>unexpected EOF</code>	پاسخ ناقص از سرور	معمولاً اپ کرش کرده؛ لاگ اپ را ببینید
<code>host.docker.internal</code> از داخل کانتینر کار نمی‌کند	شبکه Docker	در <code>docker-compose</code> از <code>host: extra_hosts</code> روی لینوکس بررسی کنید
متریک‌ها در <code>/metrics</code> هست ولی Grafana خالی است	<code>datasource/زمان</code>	بازه زمانی Grafana و <code>up</code> بودن هدف را چک کنید
<code>active_users</code> صفر می‌ماند	تازه استارت خورده	تایمر هر ۱۵ ثانیه گاج را منتشر می‌کند؛ کمی صبر کنید
<code>cpu_usage_percent</code> صفر است	اپ واقعاً بی‌کار است	زیر بار واقعی مقدار می‌گیرد؛ این درست است

۱۴. Build و انتشار پکیج

```
# Restore + Test + Pack در پوشه artifacts
.\build\pack.ps1
```

```
# یا دستی
dotnet build KSC.Observability.sln -c Release
dotnet test KSC.Observability.sln -c Release
dotnet pack KSC.Observability.sln -c Release
```

برای انتشار روی فید داخلی:

```
dotnet nuget push artifacts/*.nupkg --source <آدرس-فید> --api-key <کلید>
```

نکته دربارهٔ نسخهٔ **NET SDK**: نسخه در `global.json` پین شده تا بیلد پایدار بماند. اگر روی سیستمی چند نسخهٔ dotnet نصب است (مثلاً یک نسخهٔ x86 خراب)، از `host-64-bit` (C:\Program Files\dotnet\dotnet.exe) استفاده کنید — اسکریپت‌های `up.ps1` / `pack.ps1` این کار را خودکار انجام می‌دهند.

CI

فایل `.github/workflows/ci.yml` روی هر `push` به `main` (روی ویندوز) `restore/build/test/pack` می‌کند و پکیج‌ها و نتایج تست را به‌عنوان artifact آپلود می‌کند.

۱۵. سؤالات متداول

چرا net472؟ چون اپ‌های هدف شما .NET Framework هستند. کتابخانهٔ Abstractions روی `netstandard2.0` است تا حداکثر سازگاری داشته باشد.

آیا روی Windows Service یا WCF هم کار می‌کند؟ هستهٔ متریک (Metrics) بله؛ اما لایهٔ ASP.NET مخصوص `System.Web` است. برای `Windows Service` می‌توان مثل `KSC.Sample.SelfHost` با `PrometheusObservabilityRuntime` به‌صورت `self-host` استفاده کرد. (در صورت نیاز می‌توان یک پکیج جدا برایش ساخت.)

مدل Push می‌خواهم نه Pull. Prometheus ذاتاً `Pull` است؛ برای اپ‌های پشت فایروال می‌توان از `Pushgateway` یا `InfluxDB` استفاده کرد (تغییر در لایهٔ Metrics).

چقدر سر بار دارد؟ بسیار کم: چند گاج/شمارنده در حافظه و یک تایمر سبک. متریک‌ها فقط هنگام `scrape` سرپالایز می‌شوند.

آیا روی کاردینالیتی باید نگران باشم؟ فقط اگر `TrackRequestPath` را روشن کنید و مسیرها `id` داشته باشند. در آن صورت مسیرها را `normalize` کنید (مثلاً `/orders/{id}` به‌جای `/orders/12345`).

۱۶. ضمیمه: بازتولید پروژه از صفر (برای تیم‌های دیگر)

این بخش طوری نوشته شده که اگر فقط همین داکيومنت را در اختیار یک تیم دیگر بگذارید، بتوانند کل پروژه را از صفر بسازند. ترتیب کار دقیقاً همان ۱۰ مرحله‌ای است که پروژه ساخته شد.

۱۶.۰ پیش‌نیازها

- **NET SDK** (نسخه 8 یا 9). در `global.json` پین می‌شود.

- **NET Framework 4.7.2 Targeting Pack.** (روی ویندوز؛ برای بیلد net472).
- **Git** و یک مخزن خالی.
- **Docker Desktop** (برای استک مانیتورینگ).
- (اختیاری) **Visual Studio 2019/2022** برای اپ نمونه Web Forms.

۱۶.۱ ساختار نهایی که قرار است بسازیم

```
KSC.Observability/
├─ global.json, Directory.Build.props, NuGet.config, .gitignore, .gitattributes
├─ KSC.Observability.sln
├─ src/{Abstractions, Metrics, AspNet}
├─ tests/KSC.Observability.Tests
├─ samples/{KSC.Sample.SelfHost, KSC.Sample.WebApp}
├─ deploy/{docker-compose.yml, prometheus/, grafana/}
├─ build/pack.ps1 , up.ps1 , down.ps1 , up.cmd , down.cmd
└─ .github/workflows/ci.yml
```

مرحله ۱ — اسکلت و فایل‌های پایه

`global.json` (SDK) را پین می‌کند تا بیلد پایدار بماند):

```
{ "sdk": { "version": "9.0.314", "rollForward": "latestMinor", "allowPrerelease": false } }
```

`Directory.Build.props` (تنظیمات مشترک همه پروژه‌ها + متادیتای پکیج):

```

<Project>
  <PropertyGroup>
    <Product>KSC.Observability</Product>
    <VersionPrefix>0.1.0</VersionPrefix>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <GenerateDocumentationFile>true</GenerateDocumentationFile>
    <NoWarn>$(NoWarn);CS1591</NoWarn>
  </PropertyGroup>
  <PropertyGroup>
    <IsPackable>false</IsPackable>
    <PackageLicenseExpression>MIT</PackageLicenseExpression>
    <IncludeSymbols>true</IncludeSymbols>
    <SymbolPackageFormat>snupkg</SymbolPackageFormat>
    <PackageReadmeFile>README.md</PackageReadmeFile>
    <PackageOutputPath>$(MSBuildThisFileDirectory)artifacts</PackageOutputPath>
  </PropertyGroup>
  <ItemGroup Condition="'$(IsPackable)' == 'true'">
    <None Include="$(MSBuildThisFileDirectory)README.md" Pack="true" PackagePath=""
Visible="false" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.SourceLink.GitHub" Version="8.0.0" PrivateAssets="All" />
  </ItemGroup>
</Project>

```

سپس solution بسازید: `dotnet new sln -n KSC.Observability`

مرحله ۲ — لایه (Abstractions) Core

پروژه: `src/KSC.Observability.Abstractions` با `TargetFramework=netstandard2.0` و `RootNamespace=KSC.Observability`

— تمام تنظیمات با مقدار پیش فرض: `ObservabilityOptions.cs`

```

public sealed class ObservabilityOptions
{
    public string ServiceName { get; set; } = "dotnet-app";
    public string InstanceId { get; set; } = System.Environment.MachineName;
    public string Environment { get; set; } = "production";
    public string MetricPrefix { get; set; } = "ksc";
    public string MetricsPath { get; set; } = "/metrics";
    public bool EnableSystemMetrics { get; set; } = true;
    public bool EnableHttpMetrics { get; set; } = true;
    public bool EnableActiveUserTracking { get; set; } = true;
    public TimeSpan SystemMetricsInterval { get; set; } = TimeSpan.FromSeconds(5);
    public TimeSpan ActiveUserWindow { get; set; } = TimeSpan.FromMinutes(5);
    public bool TrackRequestPath { get; set; } = false;
    public double[] RequestDurationSecondsBuckets { get; set; }
        = { 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10 };
    public string? MetricsAccessToken { get; set; }
    public void Validate() { /* با '/'، مثبت بودن MetricsPath شروع، ServiceName بررسی خالی نبودن
        بازه ها */ }
}

```

چهار **interface** کلیدی (همگی در `KSC.Observability` namespace):

```

public interface ISystemMetricsCollector : IDisposable { void Start(); void Stop(); }

public interface IHttpMetricsRecorder {
    void RequestStarted();
    void RequestCompleted(string method, string? path, int statusCode, double elapsedSeconds);
}

public interface IActiveUserTracker : IDisposable { void Touch(string userKey); int CurrentCount {
    get; } }

public interface IObservabilityRuntime : IDisposable {
    ObservabilityOptions Options { get; }
    IHttpMetricsRecorder Http { get; }
    IActiveUserTracker Users { get; }
    void WriteMetrics(System.IO.Stream output);
}

```

`ObservabilityHost.cs` — یک نگه‌دارندهٔ سراسری (process-wide) برای runtime فعال:

```

public static class ObservabilityHost
{
    static IObservabilityRuntime? _current; static readonly object Gate = new object();
    public static bool IsInitialized => _current != null;
    public static IObservabilityRuntime Current => _current ?? throw new
InvalidOperationException("not initialized");
    public static IObservabilityRuntime? TryGet() => _current;
    public static void SetRuntime(IObservabilityRuntime rt) {
        lock (Gate) { if (_current == null) _current = rt; else rt.Dispose(); } // اولین برنده
است
    }
}

```

به علاوه ثابت‌های (service, instance, env, method, code, path, generation) `LabelNames` و `MetricSuffixes` (نام متریک‌ها بدون پیشوند).

مرحله ۳ — لایه Infrastructure (Metrics)

پروژه: `src/KSC.Observability.Metrics` با `TargetFramework=net472` ، ارجاع به Abstractions و `PackageReference: prometheus-net 8.2.1`.

نکته کلیدی محاسبه CPU (در `PrometheusSystemMetricsCollector`): یک تایمر هر چند ثانیه نمونه می‌گیرد:

```

_process.Refresh();
var nowUtc = DateTime.UtcNow;
var cpuNow = _process.TotalProcessorTime;
var wall = (nowUtc - _lastSampleUtc).TotalSeconds;
if (wall > 0) {
    var cpu = (cpuNow - _lastCpuTime).TotalSeconds;
    var cores = Math.Max(1, Environment.ProcessorCount);
    _cpu.Set(Math.Round(cpu / (wall * cores) * 100.0, 2));
}
_lastSampleUtc = nowUtc; _lastCpuTime = cpuNow;
_workingSet.Set(_process.WorkingSet64);
_managedMemory.Set(GC.GetTotalMemory(false));
_threads.Set(_process.Threads.Count);
_uptime.Set((nowUtc - _startedUtc).TotalSeconds);
for (int g = 0; g <= GC.MaxGeneration; g++)
    _gcCollections.WithLabels(g.ToString()).IncTo(GC.CollectionCount(g)); // Counter بکنواخت

```

گاج‌ها/شمارنده‌ها با `IMetricFactory` ساخته می‌شوند؛ نام = `prefix + "_" + suffix`.

— سه متریک: `PrometheusHttpMetricsRecorder`

```

_requestsTotal = factory.CreateCounter(name("http_requests_total"),
    "...", new CounterConfiguration { LabelNames = new[] { "method","code" } }); // اگر "path" فعال
_inFlight = factory.CreateGauge(name("http_requests_in_flight"), "...");
_duration = factory.CreateHistogram(name("http_request_duration_seconds"), "...",
    new HistogramConfiguration { Buckets = opts.RequestDurationSecondsBuckets, LabelNames = new[] {
"method" } });
// RequestStarted: _inFlight.Inc();
// RequestCompleted: _inFlight.Dec(); _requestsTotal.WithLabels(...).Inc();
_duration.WithLabels(method).Observe(sec);

```

— PrometheusActiveUserTracker — منطق پنجره کشویی:

```

ConcurrentDictionary<string,long> _lastSeen; // userKey -> ticks
public void Touch(string k){ if(!string.IsNullOrEmpty(k)) _lastSeen[k] = DateTime.UtcNow.Ticks; }
public void Prune(){ // صدا میزند window/4 تایمر هر
    var cutoff = DateTime.UtcNow.Ticks - _windowTicks;
    foreach (var p in _lastSeen) if (p.Value < cutoff) _lastSeen.TryRemove(p.Key, out _);
    _activeUsers.Set(_lastSeen.Count);
}

```

— PrometheusObservabilityRuntime — composition root :

```

_registry = Metrics.NewCustomRegistry(); // رجیستری ایزوله
_registry.SetStaticLabels(new Dictionary<string,string>{ // لیبل‌های ثابت روی همه متریک‌ها
    ["service"]=opts.ServiceName, ["instance"]=opts.InstanceId, ["env"]=opts.Environment });
var factory = Metrics.WithCustomRegistry(_registry);
// build_info, Http, Users و در صورت فعال بودن SystemCollector ساخته و می‌شوند Start
public void WriteMetrics(Stream s) =>
    _registry.CollectAndExportAsTextAsync(s).GetAwaiter().GetResult();

```

— ObservabilityBootstrapper — مقداردهی idempotent :

```

public static IObservabilityRuntime Initialize(ObservabilityOptions o){
    var ex = ObservabilityHost.TryGet(); if (ex != null) return ex;
    lock (Gate){ ex = ObservabilityHost.TryGet(); if (ex != null) return ex;
        ObservabilityHost.SetRuntime(new PrometheusObservabilityRuntime(o)); return
ObservabilityHost.Current; }
}

```

مرحله ۴ — لایه Integration (AspNet)

پروژه: src/KSC.Observability.AspNet با net472 ، ارجاع به Abstractions و Metrics، PackageReference: Microsoft.Web.Infrastructure 2.0.0 و Reference: System.Web, System.Configuration

ثبت خودکار ماژول — فایل PreApplicationStart.cs :


```
[assembly: PreApplicationStartMethod(typeof(ObservabilityHttpModule),
    nameof(ObservabilityHttpModule.OnPreApplicationStart))]
```

و در مازول:

```
public static void OnPreApplicationStart() =>
    Microsoft.Web.Infrastructure.DynamicModuleHelper.DynamicModuleUtility
        .RegisterModule(typeof(ObservabilityHttpModule));
```

هسته `ObservabilityHttpModule`:

```
public void Init(HttpApplication ctx){
    KscObservability.EnsureInitialized(); // web.config/ اگر مقداردهی نشده، با
    پیشفرض
    ctx.BeginRequest += OnBeginRequest;
    ctx.PostAcquireRequestState += OnPostAcquireRequestState; // در دسترس است Session اینجا
    ctx.EndRequest += OnEndRequest;
}
// OnBeginRequest: وگرنه CompleteRequest و MetricsPath → ServeMetrics == اگر مسیر
Http.RequestStarted + Stopwatch
// OnPostAcquireRequestState: می‌کند Users.Touch را (IP → نام کاربر → SessionID) کلید کاربر
// OnEndRequest: Stopwatch و Http.RequestCompleted(method, path?, status, sec) را می‌خواند و
```

سرو متریک:

```
response.ContentType = "text/plain; version=0.0.4; charset=utf-8";
runtime.WriteMetrics(response.OutputStream);
app.CompleteRequest();
```

به‌علاوه (façade) `KscObservability.Initialize(Action<ObservabilityOptions>?)` و `AppSettingsOptionsBinder` که کلیدهای `KSC.Observability:*` را از `web.config` می‌خواند.

مرحله ۵ — تست‌ها

پروژه: (net472، xUnit) `tests/KSC.Observability.Tests`. تست‌های کلیدی: اعتبارسنجی Options، شمارش/پاکسازی کاربران فعال، و خروجی exposition (شمارنده/گاج/هیستوگرام و لیبل‌های ثابت). اجرا با `dotnet test`.

مرحله ۶ — بسته‌بندی NuGet

در سه پروژه `src/*` مقادیر `IsPackable=true` و `PackageId` را ست کنید. چون پروژه‌ها به هم `ProjectReference` دارند و همگی packable اند، NuGet آن‌ها را به صورت وابستگی پکیج تبدیل می‌کند؛ یعنی نصب `KSC.Observability.AspNet` بقیه را هم می‌آورد. تولید با `dotnet pack -c Release`.

مرحله ۷ — استقرار (Prometheus + Grafana)

: deploy/docker-compose.yml

```
services:
  prometheus:
    image: prom/prometheus:v2.54.1
    ports: ["9090:9090"]
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    command: ["--config.file=/etc/prometheus/prometheus.yml", "--storage.tsdb.retention.time=30d"]
    extra_hosts: ["host.docker.internal:host-gateway"]
  grafana:
    image: grafana/grafana:11.2.0
    ports: ["3000:3000"]
    environment: { GF_SECURITY_ADMIN_USER: admin, GF_SECURITY_ADMIN_PASSWORD: admin }
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
      - grafana-data:/var/lib/grafana
    depends_on: [prometheus]
volumes: { prometheus-data: {}, grafana-data: {} }
```

دو فایل: deploy/prometheus/prometheus.yml (هدف = آپ‌های شما)، و در deploy/grafana/provisioning/ datasource (به http://prometheus:9090 با uid: PROMETHEUS) و dashboard provider (مسیر deploy/grafana/dashboards/ksc- در /var/lib/grafana/dashboards overview.json بسازید (پنل‌ها: کاربران فعال، in-flight، نرخ ریکوئست/خطا، CPU، p50/p95/p99، حافظه، thread/handle، GC با متغیر service). ساده‌ترین راه: داشبورد را در Grafana طراحی و سپس **Export** → **Save to file** کنید.

مرحله ۸ — CI (GitHub Actions)

github/workflows/ci.yml روی windows-latest (به‌خاطر net472): setup-dotnet با global-json-file: global.json، سپس pack → test → build -c Release → restore و آپلود artifact‌ها.

مرحله ۹ — نمونه‌ها

- samples/KSC.Sample.SelfHost : کنسول net472 که با TcpListener روی 0.0.0.0:9184 یک سرور HTTP کوچک می‌سازد و runtime.WriteMetrics را روی /metrics سرو می‌کند و خودش ترافیک تولید می‌کند. **نکته مهم:** قبل از بستن سوکت، کل هدرهای درخواست را بخوانید (drain) تا ویندوز به‌جای FIN یک RST نفرستد و Prometheus خطای unexpected EOF نگیرد.
- samples/KSC.Sample.WebApp : آپ کلاسیک ASP.NET Web Forms که پکیج را با PackageReference نصب می‌کند (دقیقاً مثل مصرف واقعی).

مرحله ۱۰ — اسکریپت یک دستوری

پراسس detached → انتظار تا سلامت → باز کردن مرورگر. `down.ps1` معکوسش را انجام می‌دهد. نکته: `host_۶۴`-بیتی dotnet را ترجیح دهید (`$env:ProgramFiles\dotnet\dotnet.exe`) تا با نسخه x86 احتمالی تداخل نکند.

نقشه کامیت مرحله‌ای (استاندارد، Conventional Commits)

پروژه دقیقاً با این ترتیب کامیت شد — همین الگو را دنبال کنید:

1. chore: scaffold solution structure and core abstractions
2. feat(metrics): add Prometheus-based collectors and composition root
3. feat(aspnet): add System.Web integration with auto-registered HttpModule
4. test: add unit tests for options, active users and exposition
5. build(nuget): enable packaging with symbols, readme and Source Link
6. feat(deploy): add Prometheus + Grafana stack with provisioned dashboard
7. feat(sample): add ASP.NET Web Forms sample and local NuGet feed
8. ci: add GitHub Actions build/test/pack workflow
9. docs: write full usage guide, metrics reference and changelog
10. feat: one-command launcher for the demo environment

چک‌لیست تأیید (وقتی تمام شد)

- `[ ] dotnet build KSC.Observability.sln -c Release` بدون خطا.
- `[ ] dotnet test` همه تست‌ها سبز.
- `[ ] dotnet pack -c Release` سه فایل `.nupkg` در `artifacts` می‌سازد.
- `[ ] .\up.cmd` همه چیز را بالا می‌آورد و در `http://localhost:9090/targets` هدف `up` است.
- `[ ]` در `Grafana`، داشبورد با داده پر می‌شود.

جمع‌بندی

این کار را بکنید	می‌خواهید...
<code>Install-Package KSC.Observability.AspNet</code>	یک اپ را مانیتور کنید
<code>prometheus.yml</code> و <code>.\up.cmd -NoDemo</code> و ویرایش	استک مانیتورینگ را بالا بیاورید
<code>.\up.cmd</code>	همه چیز را زنده ببینید
<code>.\build\pack.ps1</code>	پکیج بسازید
<code>http://localhost:3000</code>	داشبورد را ببینید

موفق باشید! 🚀